

Building an Agent Harness with Claude Code

A step-by-step, generally-applicable guide — scripts, skills, and components included

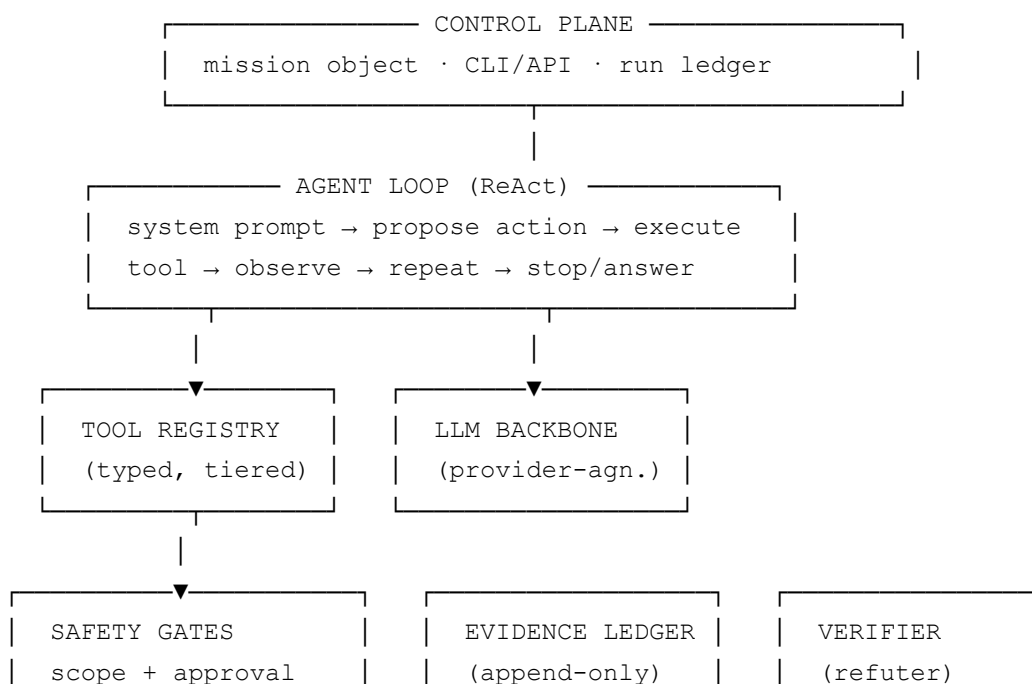
Audience: dual — written to be read by a human *and* executed by an agent (Claude Code).

Target: a harness for a placeholder objective, written throughout as `[OBJECTIVE]` (e.g. "audit a codebase for X", "triage support tickets", "run recon on authorized targets", "grade student submissions"). Substitute your own goal wherever `[OBJECTIVE]` appears. **Stack:** TypeScript + Node (the patterns are language-agnostic; Python notes included). **Design lineage:** the tiered architecture (BASIC → INT → ADV) from the companion *How-to-Harness* report — the model is the brain, the harness is discipline.

How to use this document as an agent: each Phase has a ► **CLAUDE CODE PROMPT** block (paste it into Claude Code verbatim) followed by the **artifact(s)** that prompt should produce. Build strictly top-to-bottom; do not skip the safety gates in Phase 1.4 and Phase 2.2.

0. What you are building

A harness is the deterministic scaffold around a non-deterministic model:



You will build this in three passes:

- **BASIC** (Phase 1): one agent, one loop, one tool set, safe and reproducible.
- **INT** (Phase 2): specialists, human-in-the-loop approval, an evidence ledger, a verifier.
- **ADV** (Phase 3): multi-model orchestration, self-benchmarking, human-gated actions, a lessons loop.

Plus two cross-cutting deliverables that make it consumable:

- **Skills** (Phase 4): reusable `SKILL.md` capabilities so Claude Code can extend the harness on demand.
 - **Consumability layer** (Phase 5): `CLAUDE.md` , slash commands, subagents, MCP.
-

1. Phase 1 — BASIC harness

1.0 Prerequisites & project init

```
# Claude Code (agent CLI). Requires Node 18+.
npm install -g @anthropic-ai/claude-code    # verify current install steps in offi
node --version                               # >= 18

mkdir harness-[objective] && cd $_
git init
npm init -y
npm i eventemitter3 zod
npm i -D typescript tsx @types/node eslint
npx tsc --init
```

► **CLAUDE CODE PROMPT (kickoff — establishes the working agreement):**

```
You are helping me build an agent harness for [OBJECTIVE].
Read ./CLAUDE.md if it exists, otherwise we'll create it.
Constraints for everything you write:
- TypeScript, ESM, strict mode. Small, pure, unit-testable modules.
- Every side-effecting tool must pass a safety gate before executing.
- No network calls or file writes outside the project without an explicit approva
- Produce a matching *.test.ts for each module.
Confirm you understand, then wait for the first task.
```

Set up the directory contract Claude Code will fill in:

```
src/
  llm/          # provider abstraction          (1.1)
  config/       # config + keys                (1.2)
  agent/        # ReAct loop                   (1.3)
  safety/       # scope + approval gates       (1.4, 2.2)
  tools/        # typed tool registry          (1.5)
  parse/        # robust output parsing        (1.6)
  mission/      # the mission object           (1.7)
scripts/       # verify, bench, doctor        (2.7, 3.2)
skills/        # SKILL.md capabilities         (Phase 4)
.claude/       # commands, subagents          (Phase 5)
```

1.1 Provider abstraction (the model behind one interface)

► CLAUDE CODE PROMPT :

Create `src/llm/index.ts` exposing an `LLMBackbone` class with a single `complete()` method that normalizes across providers. Requirements:

- Config: { provider, model, apiKey?, baseUrl?, maxTokens, temperature }.
- Providers: 'anthropic' | 'openai' | 'openrouter' | 'local' (OpenAI-compatible,
- Support a text-mode tool-calling shim: if a model lacks native tool-calling, serialize tool schemas into the system prompt and parse tool calls back out of
- Retry with exponential backoff; a `'fallback'` list of configs tried in order.
- No secrets logged. Return { text, toolCalls[], usage }.

Write `src/llm/index.test.ts` with a fake transport (no real network).

Artifact (skeleton the prompt should yield):

```
// src/llm/index.ts
export type Provider = 'anthropic' | 'openai' | 'openrouter' | 'local';

export interface LLMConfig {
  provider: Provider;
  model: string;
  apiKey?: string;
  baseUrl?: string;
  maxTokens?: number;
  temperature?: number;
}

export interface ToolCall { name: string; input: Record<string, unknown>; id: str
export interface Completion { text: string; toolCalls: ToolCall[]; usage: { in: n
```

```
export class LLMBackbone {
  constructor(private cfg: LLMConfig, private fallbacks: LLMConfig[] = []) {}

  async complete(messages: {role: string; content: string}[], tools?: ToolSchema[]) {
    for (const cfg of [this.cfg, ...this.fallbacks]) {
      try { return await this.call(cfg, messages, tools); }
      catch (e) { /* backoff + continue to next fallback */ }
    }
    throw new Error('all providers failed');
  }
  // call() dispatches per provider; local/openrouter/openai share the OpenAI wir
}
```

Why this first: the provider seam is what makes the harness keyless-capable and testable. A `local` provider means you can iterate the loop for free before spending a token on a hosted model.

1.2 Config + key management

► CLAUDE CODE PROMPT :

Create `src/config/index.ts`: a typed `Config` loaded from `env` + an optional `~/.harness/config.json`, with a ``defaultProvider``, an `AVAILABLE_MODELS` registry, and `env-var` key resolution (never hardcode keys). Provide `loadConfig()` and a `redact()` helper that masks secrets in any object before logging. Add tests.

1.3 The ReAct agent loop (dumb loop, smart model)

► CLAUDE CODE PROMPT :

Create `src/agent/index.ts`: an `Agent` class running a ReAct loop.

- Inputs: `LLMBackbone`, `ToolRegistry`, `system prompt`, a `Mission`, `maxSteps`.
- Each step: ask model for next action → if it's a tool call, run it THROUGH the safety gate → feed the observation back → repeat. Stop on an explicit final answer or when `maxSteps` is hit (hard ceiling, non-negotiable).
- Emit typed events (`step`, `toolCall`, `observation`, `done`, `error`) via `eventemitter3` so a UI/CLI can subscribe.
- Every tool result and model turn is written to the `Mission`'s append-only log.

Add `src/agent/index.test.ts` driving a fake LLM through a 3-step scripted run.

Artifact (the loop's core):

```
// src/agent/index.ts (core)
async run(): Promise<AgentResult> {
```

```

for (let step = 0; step < this.maxSteps; step++) {
  const out = await this.llm.complete(this.mission.transcript(), this.registry.
this.mission.record({ role: 'assistant', content: out.text }));
  if (out.toolCalls.length === 0) return this.finalize(out.text); // model ch
  for (const call of out.toolCalls) {
    const gate = await this.safety.check(call, this.mission); // 1.4 / 2
    if (!gate.allowed) { this.mission.record({ role: 'tool', content: `DENIED:
const obs = await this.registry.execute(call);
    this.mission.record({ role: 'tool', content: obs, provenance: call });
  }
}
return this.finalize('[max steps reached]');
}

```

1.4 Safety gate — scope containment (the non-negotiable floor)

► CLAUDE CODE PROMPT :

Create src/safety/scope.ts. A pure function checkScope(call, mission) that, for any tool touching the outside world (network, filesystem writes, shell), allows it ONLY if the target is inside the mission's declared scope. Deny off-scope hosts/paths, loopback, and private ranges by default with a clear reason string ("SCOPE DENIED: <detail>"). No I/O in this module — it's a pure predicate so it's fully unit-testable. Add exhaustive tests including edge cases.

This belongs in BASIC, not later. A harness that can act before it can refuse is a liability. Scope is declared on the mission (1.7) and enforced here on every side-effecting call.

1.5 Typed tool registry

► CLAUDE CODE PROMPT :

Create src/tools/registry.ts. A ToolRegistry where each tool is:

- { name, description, inputSchema (zod), riskTier: 'safe'|'active'|'intrusive'|'cr
- register(tool), schemas() (for the model), execute(toolCall) which validates input against the zod schema THEN runs.
- Ship 2-3 inert 'safe' starter tools appropriate to [OBJECTIVE] (e.g. read_file, http_get (scope-checked), list_dir). No dangerous tools yet. Add tests. Keep the registry decoupled from the loop and the gate.

Artifact (tool shape):

```

// src/tools/registry.ts
export interface Tool<I = unknown> {

```

```

    name: string;
    description: string;
    inputSchema: z.ZodType<I>;
    riskTier: 'safe' | 'active' | 'intrusive' | 'credential' | 'dangerous';
    run(input: I): Promise<string>;
  }

```

1.6 Robust output parsing

► CLAUDE CODE PROMPT :

Create `src/parse/json.ts` with `parseLastObject(text)` and `parseLastArray(text):` strip ``` fences, try a direct parse, then scan every balanced {...} / [...] span and return the LAST one that parses to the expected shape (the model's final answer). Never trust a naive `JSON.parse` on model output. Add tests with fenced, trailing-p and stray-brace inputs.

1.7 The mission object (seed of the control plane)

► CLAUDE CODE PROMPT :

Create `src/mission/index.ts`. A `Mission` holds: `id`, `objective ([OBJECTIVE])`, `scope` (allowed hosts/paths), `status`, an append-only transcript, and a findings array. Methods: `record()` (immutable append), `transcript()` (for the model), `addFinding()`, `snapshot()` (serializable). Findings and transcript entries are never mutated after write. Add tests asserting append-only behavior.

 **End of BASIC:** you now have a single agent that reasons, calls scope-checked tools, parses robustly, and logs everything immutably. Run it end-to-end against a trivial `[OBJECTIVE]` before proceeding.

2. Phase 2 — INT harness (coordination, human-in-the-loop, trust)

2.1 Specialist loadouts (compose, don't fork)

► CLAUDE CODE PROMPT :

Create `src/agent/specialists.ts`. A `Specialist = { role, systemPrompt, toolAllowli`
 Define a `Loadout = { specialists[], toolRegistrySubset, targetAdapter, benchmark?`
 Refactor `Agent` to accept a `Specialist` so I can run role-scoped agents

(e.g. for [OBJECTIVE]: a 'collector', a 'analyst', a 'reporter') that share the same loop but differ in prompt + tool subset. New domains must compose as a new Loadout, never a fork of the loop. Add a registry of loadouts. Tests included.

2.2 Approval gate + risk tiers (human veto with fail-safe-deny)

► CLAUDE CODE PROMPT :

```
Create src/safety/approval.ts (PURE – host injects the approver + warning sink).
- GATED_TIERS = {intrusive, credential, dangerous}: a tool in these tiers is INER
  until approved. Approve-once-then-free per session.
- SPICY_TIERS = {credential, dangerous}: even when approved, emit a loud audited
  warning on EVERY call.
- Two approval paths: interactive (ask the human on first use) and pre-authorized
  (headless runs get an allowlist up front; off-list denies; everything audited).
- FAIL-SAFE: a gated tool with no approver wired and not pre-approved is DENIED.
Compose this with checkScope so Agent calls one safety.check(). Tests with fakes.
```

Artifact (composition):

```
// src/safety/index.ts
export async function check(call, mission, policy): Promise<{allowed: boolean; re
  const scope = checkScope(call, mission);           // 1.4
  if (!scope.allowed) return scope;
  return requestApproval(call, policy);                // 2.2 (fail-safe denies)
}
```

2.3 Evidence ledger + provenance gate

► CLAUDE CODE PROMPT :

```
Create src/evidence/gate.ts. A Finding must carry provenance (which tool call +
which observation produced it). evidenceGate(finding) REJECTS any finding not
backed by a real tool observation in the mission log – this is the antidote to
model confabulation. addFinding() routes through the gate. Append-only. Tests.
```

2.4 Adversarial verification (the refuter)

► CLAUDE CODE PROMPT :

```
Create scripts/verify-finding.mjs and src/evidence/refuter.ts.
Before a finding is promoted to "confirmed", a SEPARATE model instance (the
refuter, ideally a different model/temperature) is prompted to disprove it using
only the committed evidence. A finding that survives refutation → confirmed;
```

one that doesn't → retracted with the refuter's reason logged. This second-model check is the highest-leverage quality mechanism in the harness. Add a test that feeds a known-false finding and asserts it gets retracted.

2.5 Control plane — CLI + mission lifecycle

► CLAUDE CODE PROMPT :

```
Create src/cli.ts (tsx entry) and src/server.ts (optional HTTP API).
CLI verbs: `start "<objective>" --scope <...>`, `status <id>`, `report <id>`.
HTTP: POST /api/mission/start, GET /api/mission/status. A natural-language front
door: accept a plain-English objective, resolve it into a Mission + Loadout, laun
Stream agent events to stdout / SSE. Tests for the lifecycle transitions.
```

2.6 Reproducibility guard (receipts, not vibes)

► CLAUDE CODE PROMPT :

```
Create scripts/verify-claims.mjs. It re-derives every headline metric the harness
reports from committed artifacts under ./bench (JSON), and exits non-zero if any
number can't be reproduced. Wire it as `npm run verify-claims` and add a git
pre-push hook (.github/hooks/pre-push) that runs it. Principle: a claim that can't be
reproduced doesn't ship.
```

Artifact (`package.json` scripts to mirror):

```
{
  "scripts": {
    "dev": "tsx src/cli.ts",
    "build": "tsc",
    "doctor": "node scripts/doctor.mjs",
    "verify-claims": "node scripts/verify-claims.mjs",
    "verify-finding": "node scripts/verify-finding.mjs",
    "bench": "node scripts/bench.mjs",
    "test": "node --test"
  }
}
```

2.7 Health check (`doctor`)

► CLAUDE CODE PROMPT :

```
Create scripts/doctor.mjs: verifies Node version, that a provider is reachable
```

(or a local model is up), that required tools exist on PATH, and that config loads without exposing secrets. Human-readable / output. Non-zero exit on fai

 **End of INT:** the harness now runs role-scoped specialists, gates dangerous actions behind a human with fail-safe-deny, refuses unprovenanced findings, refutes its own conclusions, and re-derives its own numbers.

3. Phase 3 — ADV harness (orchestration, benchmarking, self-improvement)

3.1 Multi-model orchestrator (use only the benign pattern)

► **CLAUDE CODE PROMPT :**

Create `src/orchestration/orchestrator.ts`. A master-builder that decomposes a large [OBJECTIVE] into independent sub-queries, dispatches them to worker agents (possibly cheaper models) in parallel with bounded concurrency, and synthesizes results over rounds with an accumulated-knowledge memory. Include context-packing to keep each worker under a token budget.

IMPORTANT — scope discipline: this is a WORKLOAD-DECOMPOSITION pattern only (parallelism, cost, context budgeting). Do NOT design it to conceal the objective from, or route around the safety behavior of, any worker model. Each worker sees honest description of its subtask. Add tests for round synthesis + concurrency li

The decomposition pattern is genuinely powerful for parallelism and cost. Keep it honest: workers get a truthful description of their subtask. Do not build context-isolation intended to defeat a worker model's guardrails — that is an anti-pattern (and gets detected and blocked by providers).

3.2 Benchmark harness with committed oracles

► **CLAUDE CODE PROMPT :**

Create `scripts/bench.mjs` and `./bench/<suite>/` with tasks + a committed ground-truth oracle per task. The runner grades each result against the oracle (never a self-report), computes pass@1 with a Wilson-95 interval, and writes results as committed JSON that verify-claims re-derives. Add an ANTI-FITTING guard: fail the build if grading logic references the specific test answers, and keep held-out tasks separate from any tasks used to tune prompts. Report black-box and white-box variants separately; never blend them.

3.3 Human-gated terminal actions

► **CLAUDE CODE PROMPT :**

For any consequential terminal action in [OBJECTIVE] (sending, publishing, deleting, deploying, disclosing), the harness must STOP one step short and produce a draft/plan that a human explicitly approves and executes. Implement a ``requireHumanRelease(action)`` checkpoint. Add a test asserting the harness never auto-executes a terminal action.

3.4 Lessons / self-improvement loop

► **CLAUDE CODE PROMPT :**

Create `scripts/lessons.mjs + src/resources/lessons.ts`. After each mission, extract structured lessons (what worked, what failed, prompt/tool gaps) into a committed store. Provide `retrieveLessons(objective)` so future mission planning can load relevant past lessons into context. Mark clearly which part is live (recording) vs. roadmap (auto-feedback into planning). Tests for record + retrieve.

✅ **End of ADV:** the harness plans across models honestly, benchmarks itself against committed oracles with an anti-fitting guard, refuses to auto-fire consequential actions, and accumulates lessons.

4. Phase 4 — Skills (make the harness self-extending)

Claude Code Skills are `SKILL.md` files that teach the agent a repeatable capability. Create a `skills/` folder; each skill is a directory with a `SKILL.md` (front-matter + instructions) plus optional helper scripts. Build these three so the harness can extend itself.

4.1 Skill: `harness-tool-adder`

`skills/harness-tool-adder/SKILL.md :`

```
---
name: harness-tool-adder
description: >
  Use when the user wants to add a new tool to the harness's ToolRegistry.
  Triggers: "add a tool", "give the agent the ability to X", "new capability".
---
# Adding a tool to the harness
```

1. Read `src/tools/registry.ts` to learn the Tool interface and existing patterns.
 2. Create `src/tools/<name>.ts` implementing Tool: name, description, zod inputSchema, riskTier (choose conservatively – anything with side effects is at least 'action', anything intrusive/credential/dangerous MUST be gated), and `run()`.
 3. If the tool touches network/fs/shell, confirm `checkScope` covers it; add a case.
 4. Register it in `src/tools/index.ts`.
 5. Write `src/tools/<name>.test.ts` (happy path + a scope-denied path).
 6. Run ``npm test``. Do not mark done until green.
- NEVER register a dangerous tool without a riskTier that forces the approval gate.

4.2 Skill: harness-eval-runner

skills/harness-eval-runner/SKILL.md :

```
---
name: harness-eval-runner
description: >
  Use to add or run a benchmark suite for the harness. Triggers: "benchmark",
  "evaluate the harness", "add an eval", "measure pass@1".
---
# Running / extending harness evals

1. Read scripts/bench.mjs for the runner contract.
2. To add a suite: create bench/<suite>/tasks.json and bench/<suite>/oracle.json
   (committed ground truth). Keep held-out tasks in a separate file.
3. Grade only against the oracle – never a model self-report.
4. Run `npm run bench`, then `npm run verify-claims` to confirm the numbers re-de
5. If verify-claims fails, the eval is not trustworthy – fix before reporting any
```

4.3 Skill: harness-refuter

skills/harness-refuter/SKILL.md :

```
---
name: harness-refuter
description: >
  Use to verify a finding before it is reported as confirmed. Triggers:
  "verify this finding", "is this real", "confirm before reporting".
---
# Refuting a candidate finding

1. Load the finding and its provenance from the mission log.
2. Spawn a refuter (different model or temperature) prompted to DISPROVE the find
   using ONLY the committed evidence.
3. Survives refutation → mark confirmed. Fails → retract, logging the refuter's r
```

4. Never report a finding as confirmed if it lacks provenance (evidenceGate) or was retracted. Confabulation is the default failure mode; this step is the guard.

Skill design rules: keep the `description` trigger-rich (that's what routes the agent to it), keep steps imperative and verifiable, and end with a hard "don't mark done until..." so the agent can't declare victory early.

5. Phase 5 — Consumability layer (agent + human)

5.1 `CLAUDE.md` (the project's standing instructions)

Create `CLAUDE.md` at the repo root — Claude Code reads it automatically on every session.

```
# Harness for [OBJECTIVE]

## What this is
A tiered agent harness (BASIC/INT/ADV). The model reasons; the harness enforces
safety, provenance, and reproducibility.

## Non-negotiable rules for any change
1. Every side-effecting tool passes checkScope + the approval gate. Never bypass.
2. Findings without provenance are rejected (evidenceGate). Never report unverifi
3. Consequential terminal actions require requireHumanRelease — never auto-execut
4. Any headline number must re-derive via `npm run verify-claims` (runs on pre-pu
5. New capabilities compose as a Loadout/Tool, never a fork of the agent loop.

## Layout
src/llm, src/agent, src/tools, src/safety, src/evidence, src/mission, src/orchest
scripts/ (verify-claims, verify-finding, bench, doctor, lessons)
skills/ (harness-tool-adder, harness-eval-runner, harness-refuter)

## Commands
npm run dev · npm test · npm run verify-claims · npm run bench · npm run doctor
```

5.2 Slash commands (`.claude/commands/`)

Create reusable prompts as slash commands. Example `.claude/commands/add-tool.md`:

```
Add a new tool named "$ARGUMENTS" to the harness using the harness-tool-adder ski
Pick the correct riskTier, wire the safety gate, register it, and add tests. Repo
riskTier you chose and why before finishing.
```

Then in Claude Code: `/add-tool http_post .`

Other useful commands: `/verify <finding-id>`, `/bench <suite>`, `/new-loadout <domain> .`

5.3 Subagents (`.claude/agents/`)

Define role-scoped subagents so Claude Code can dispatch specialists. Example

`.claude/agents/refuter.md`:

```
---
name: refuter
description: Adversarially verifies a candidate finding using only committed evid
tools: Read, Grep
---

You are a skeptic. Given a finding and its provenance, try to DISPROVE it using o
the evidence in the mission log. If you cannot disprove it, say CONFIRMED with th
specific evidence lines. If you can, say RETRACT with the reason. Never speculate
beyond the committed evidence.
```

5.4 MCP exposure (let other agents call your harness)

► CLAUDE CODE PROMPT :

```
Create src/mcp-server.ts exposing a safe subset of the harness over MCP
(e.g. a `run_[objective]` tool that starts a scoped mission and returns status).
Only expose non-dangerous, scope-gated capabilities. Document the tool schema.
```

6. Build order checklist (for an agent to execute)

```
[ ] 0    project init, CLAUDE.md, directory contract
[ ] 1.1 llm backbone + fallback + tests
[ ] 1.2 config + redact + tests
[ ] 1.4 scope gate (pure) + tests          ← before any real tool
[ ] 1.5 tool registry + 2 inert tools + tests
[ ] 1.6 robust parser + tests
[ ] 1.7 mission (append-only) + tests
[ ] 1.3 ReAct loop wiring + end-to-end smoke on a trivial [OBJECTIVE]
[ ] 2.2 approval gate + fail-safe-deny + tests
[ ] 2.1 specialists + loadouts + tests
[ ] 2.3 evidence gate (provenance) + tests
[ ] 2.4 refuter + verify-finding + tests
```

```
[ ] 2.5 cli + lifecycle + tests
[ ] 2.6 verify-claims + pre-push hook
[ ] 2.7 doctor
[ ] 3.1 orchestrator (honest decomposition) + tests
[ ] 3.2 bench + oracles + anti-fitting guard
[ ] 3.3 requireHumanRelease checkpoint + test
[ ] 3.4 lessons record/retrieve + tests
[ ] 4   three SKILL.md capabilities
[ ] 5   CLAUDE.md, slash commands, subagents, MCP
[ ] ✓   npm test green · npm run verify-claims green · npm run doctor green
```

7. Safety & integrity invariants (never violated)

These are the load-bearing rules distilled from the architecture. Bake them into `CLAUDE.md`, tests, and CI:

1. **Scope before action.** No side-effecting tool runs outside declared scope. (1.4)
 2. **Fail-safe deny.** A gated tool with no approver, unattended, is denied. (2.2)
 3. **Provenance before claim.** No finding without a real observation behind it. (2.3)
 4. **Refute before confirm.** A second model must fail to disprove it. (2.4)
 5. **Human before consequence.** Terminal actions stop one step short. (3.3)
 6. **Reproduce before report.** Every number re-derives from committed artifacts. (2.6, 3.2)
 7. **Honest decomposition.** Workers get a truthful subtask; no evasion-by-isolation. (3.1)
 8. **Compose, don't fork.** New domains are loadouts, not loop rewrites. (2.1)
-

8. Adapting to your [OBJECTIVE]

The harness is domain-neutral. To specialize it, change only:

- **Tools** (`src/tools/`) — the capabilities the objective needs.
- **Specialists/loadout** (`src/agent/specialists.ts`) — the roles and prompts.
- **Target adapter** — how the mission's target is described and scoped.
- **Benchmark** (`bench/`) — the oracle-graded tasks that prove it works.

Everything else — the loop, gates, ledger, refuter, verifier, orchestrator, control plane — is

reused unchanged. That reuse *is* the meta-harness. If your objective involves acting on systems or data you don't own, the scope gate, approval gate, and human-release checkpoint are not optional — they are the difference between a tool and a hazard.

Built to be executed top-to-bottom by Claude Code and read by a human. The model is the brain; the harness is discipline — and the discipline is the safety gates, the provenance, and the receipts.